

CENG-322

Deliverable 5

FINAL DOCUMENT LAYOUT

Please submit a **single PDF document per group**. This means that the document should be professionally organized and have a uniform style throughout. It should look as though it came from one team, not 4 separate students. Please note that instructors may choose to run your submission through TurnItIn or compare the submission with other students from other sections for academic integrity purposes.

A single **PDF document** with the following sections denoted using page numbers, headers/footers and a table of contents:

All team members must present, missing members will be assigned 0.

Follow the steps as requested below, meaning order your content as below.

Use proper formatting

1. Document Title: Deliverable 5
2. Team Name: LIFE (*Light, Indoor air, Freshness, and Energy*)
3. Project Name: LIFE Environmental Solutions
4. Student's names and IDs
5. Table of Contents
6. Brief description of the project, couple sentences. **We want to be able to have LIFE detect human beings in an indoor space (warehouse, offices, living rooms) and be able to alert when harmful contents are in the air. We also want to have LIFE able to detect any harmful gases like CO2 and be able to warn us so safety is an important thing in our product.**
7. Table below with signatures 8. Enter **github** Id into the table.
9. Members Info and Participation. Effort was put on the project (0 – 100%), students will be marked based on their effort, i.e. if the team gets 8,

and one student effort was only 50%, that student mark will be adjusted to $8 * 50\% = 4$

Name	Student Id	Github Id	Signature	Effort
Mohamed Ali	N01440760	MohamedAli0760	M.A.A.	100%
Jonathan Akabuike	N01510573	JonathanAkaBuike0573	J.A	100%
Farhan Habibzai	N01610299	FarhanHabibzai	F.H.	100%

10. In the pdf, write the requirement and comment on it for each item. Include the question and the answer.
11. GitHub Repo link. All members must contribute to the repo.
12. Verify the link is working.
GitHub Link:
https://github.com/JonathanAkabuike0573/LIFE_EnvironmentalSolutions.git
13. I will use the following credentials to test your app: Email: aaa@bbb.com
Password: *Admin101!*

Also include any other account needed to test, i.e. Admin, customer, ...etc.
14. Each member must have a minimum of 20 commits. Counted starting **Nov. 17**.
15. Marks deducted for members who are not meeting the minimum 20 commits (i.e. if only 16 commits, 20% percent deducted, ...etc.), the commits should happen for a duration of minimum of 8 days!
16. Each member must have a minimum of 8 days of commit (Nov. 17 – Nov. 30), otherwise marks deducted.
17. List number of commits for each member for this sprint only. Create a table like below:

Mohamed	Jonathan	Farhan	
Nov. 23, 1 commit	Nov. 23, 6 commits		
Nov 24, 3 commits	Nov. 24, 2 commits		
Nov 25, 1 commit	Nov. 25, 2 commits		
Nov 26, 11 commits	Nov. 27, 2 commits		
Nov 27, 1 commit	Nov. 28, 4 commits		
Nov 28, 4 commits	Nov. 29, 3 commits		
Nov 29, 1 commit	Nov 30,		
Nov 30, 4 commits			
Total: 26 commits			

18. Sprint 5 goals, list sprint goals and not tasks you worked on.:

Sprint 5 Goals

1. Deliver Intelligent Smart Building Automation
Enable the system to actively respond to environmental changes by implementing automated lighting control based on occupancy and configurable safety alerts for air quality hazards.
2. Implement Comprehensive Security Monitoring
Establish an "After-Hours" surveillance capability that detects unauthorized presence during specified time windows and triggers immediate user notifications.
3. Establish a Unified Notification & History System
Create a centralized event log that captures and persists all critical system alerts (security breaches, air quality warnings) into a database-backed history feed for user review.
4. Finalize Sensor Data Integrity & Real-Time Sync
Ensure accurate, bi-directional synchronization between the mobile app and Firebase for all sensor modules (Light, Air Quality, Presence) to support reliable multi-room monitoring.
5. Prepare Application for Monetization & Deployment
Lay the architectural groundwork for In-App Purchases (premium upgrades) and conduct final code readiness checks for Google Play Store submission.

19. Describe 3 tasks from the Scrum dashboard that are related to addressing technical debt.:

The transition to MVVM (Repositories/ViewModels) was foundational work that enabled the specific feature tasks in your Epic list. While the Epic list focuses on *features* ("Smart Lights", "Alerts"), the underlying work to make those features possible involved this architectural cleanup.

1. Repository Control & Data Separation maps to Task 1.4

- *Task:* "Update LightRepository to allow external control (fix null ref issue) so PresenceRepository can command lights."
 - *Connection:* You couldn't have done this cleanly without the Repository pattern. By having a LightRepository, you created a centralized place that PresenceRepository could call to turn off lights, decoupling the "Presence Logic" from the "Light UI".
2. ViewModel Logic for Alerts maps to Task 3.1 & 3.3
- *Task:* "Update AirQualityViewModel to monitor incoming eCO2 data..." & "Connect AirQualityViewModel to NotificationHelper..."
 - *Connection:* Instead of writing this alert logic inside AirQualityFragment (which would stop working if the user navigated away), you put it in AirQualityViewModel. This ensures the monitoring and alerting logic lives in a lifecycle-aware component separate from the UI.
3. Centralized Data Listening maps to Task 4.2 & 4.4
- *Task:* "Create NotificationRepository to listen to Firebase..." & "Refactor NotificationViewModel to remove dummy data..."
 - *Connection:* This is the definition of the Repository pattern. You moved the data fetching (NotificationRepository) away from the UI presentation (NotificationViewModel), allowing the list to update dynamically without the Fragment knowing *how* the data is fetched.

20. Describe in detail, the work that has been completed by each team member in this sprint only.

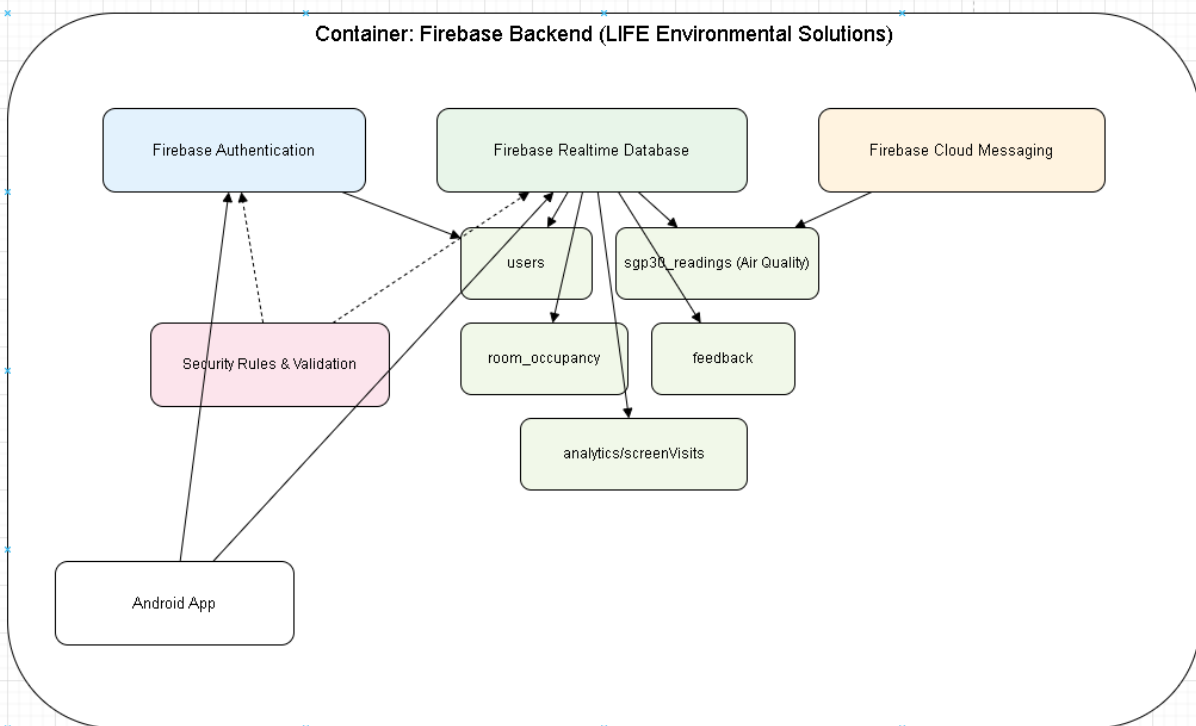
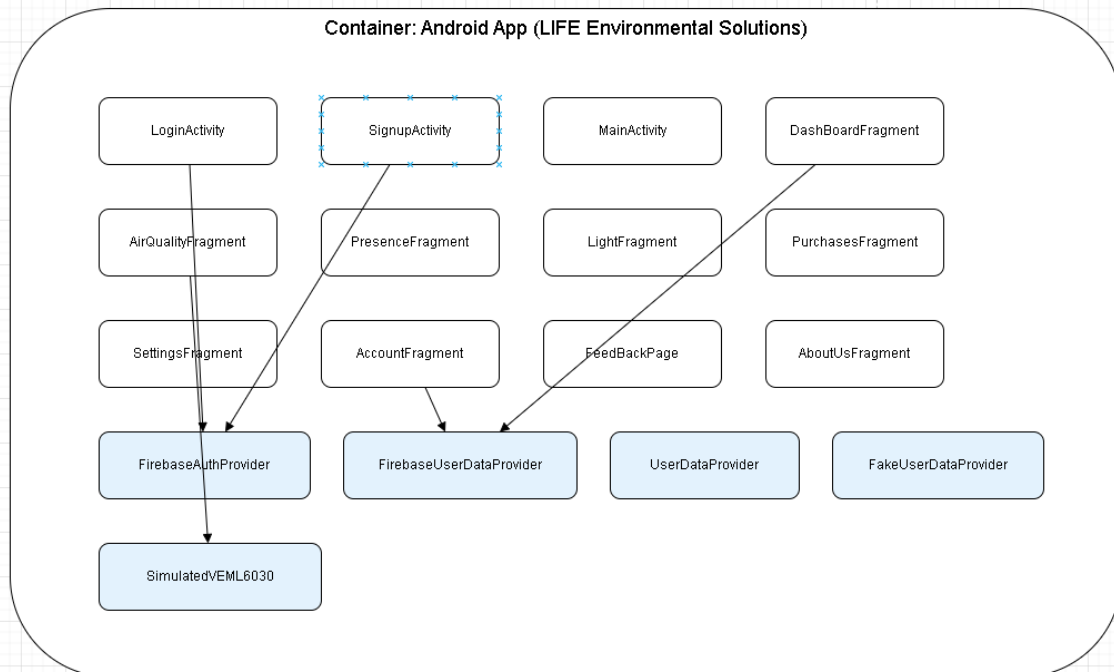
Mohamed: I created the Energy Fragment so that we can get the readings from the DB for the ina 219 sensor readings. Updated the purchases page so that the user can select what type of payment which plan and the options to that plan. Also included in the email receipt option and cancel upgrade option

Jonathan: I implemented the Intelligent Automation & Alert System to make the app actively respond to sensor data. I refactored the code into an MVVM architecture (Repositories & ViewModels), which separated the logic from the UI and fixed data synchronization issues.

I developed the Smart Lighting logic so lights only turn off when a room is truly vacant and the light is actually on. I also built the Security and Air Quality Alert system, which sends notifications for after-hours occupancy or high CO2 levels, and created the Notification History feature to save and display these alerts. Additionally, I separated the Account Management features from the general Settings to improve navigation and user flow

Farhan: I implemented the Fab button so that we can see the wifi before and after. I took screenshots showing the snackbar alert dialog, the notification and toast, along with the C4 model component diagram showcasing the Android App and the FireBase BackEnd.

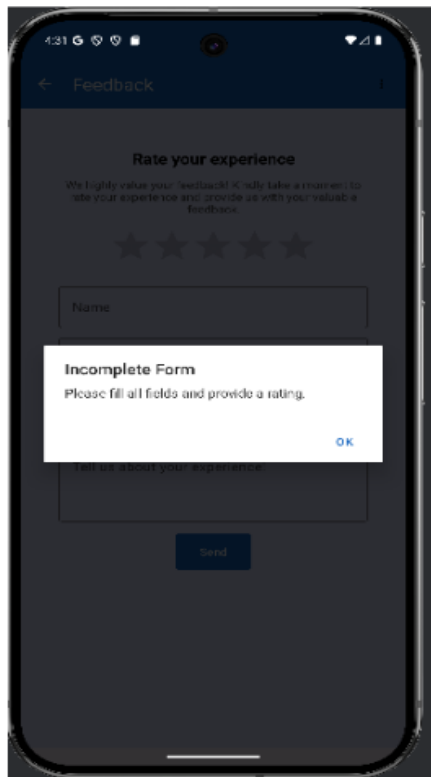
21. Using C4 Model, draw “Component Diagram”. Choose two of the containers in your system. Draw two detailed components diagram.



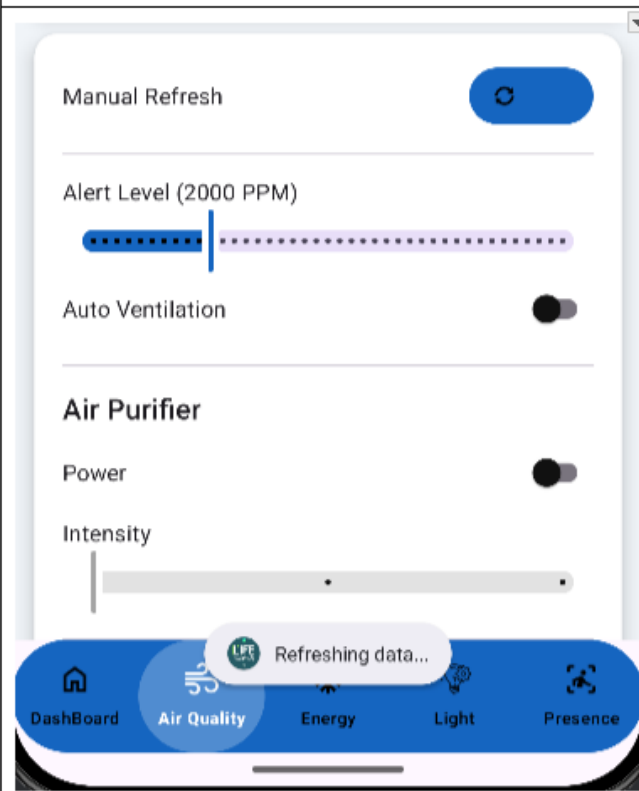
22. Use a tool to draw your diagrams, hand drawing will not be accepted. I suggest you use <https://app.diagrams.net/>
23. Demonstrate the use of all of the following in your app, in the appropriate places: Include screenshots to show.
 - A. AlertDialog
 - B. Snackbar
 - C. Toast
 - D. Notification, handle runtime for API 33 and higher.

Screenshots for the alertdialog, snackbar, toast and notification

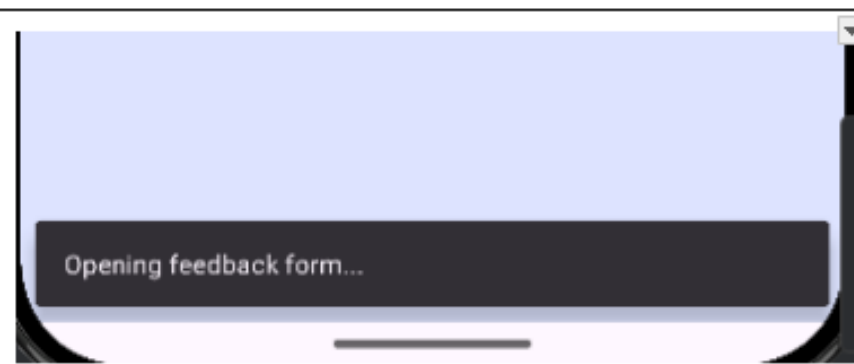
AlertDialog



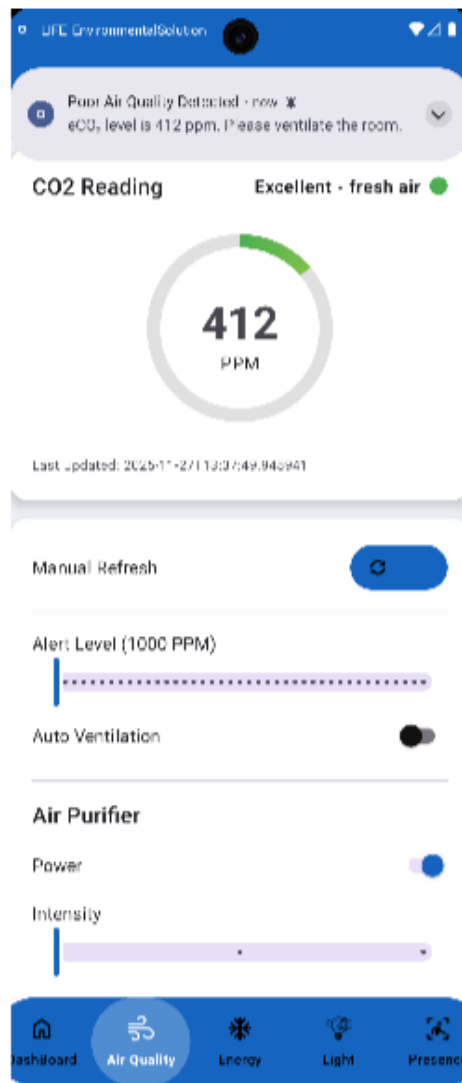
Toast



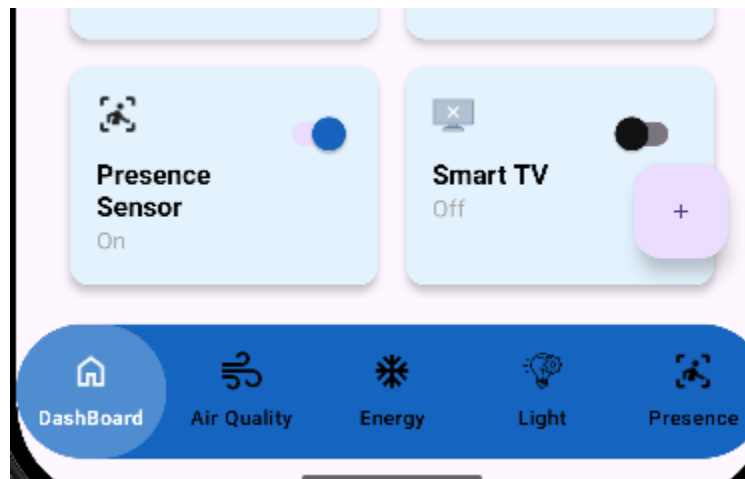
Snackbar



Notification

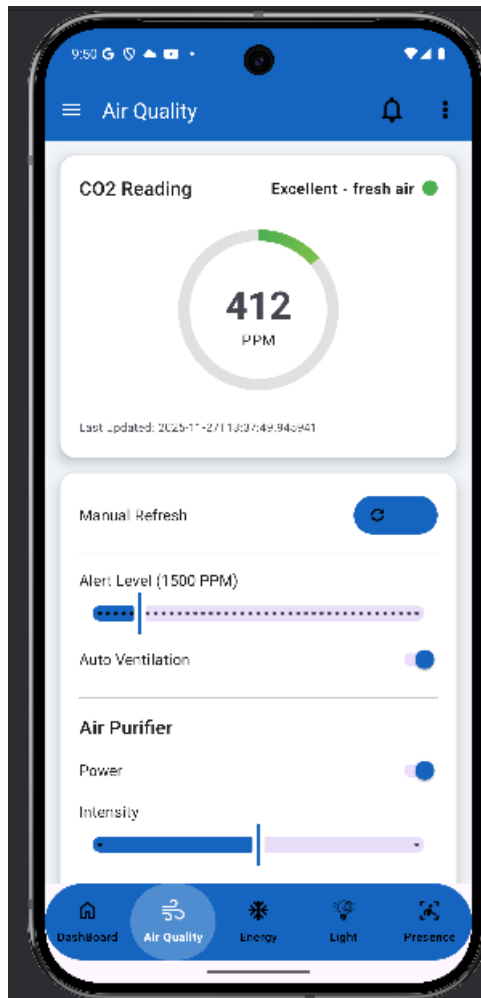


24. Screenshot showing the fab button and how the functionality fits into your app.

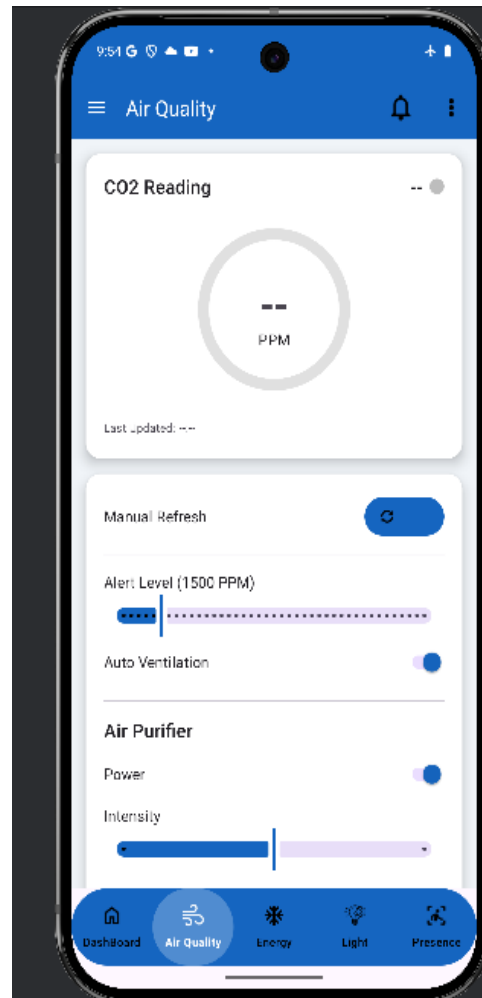


25. Should implement some functionality for off-line mode. Document what feature will work off-line. Should be a meaningful feature. Will be tested, when in Airplane mode. Provide screenshot.

Before (With Wi-Fi Still on)



After (With Wi-Fi is off)



26. Describe how you handle exceptions, and list the exceptions that you had to handle.
Exceptions are handled by wrapping risky code blocks in try blocks. If the error occurs, then the catch block intercepts the exception, allowing the app

to either log the error, provide a fallback value, or show a user-friendly message instead of crashing.

The `ParadeException` is in the `PresenceViewModel.java`. It is Encountered twice in this java file. This typically occurs when parsing the dates or specific data formats. Handling ensures that if the date string from a sensor or API is malformed, the app can still function without that specific data point

The `Generic Exception` is in the `AirQualityViewModel.java`. A general catch-all for unexpected errors is likely during the data processing or initialization of the view model. This acts as a safety net for any other unforeseen runtime issues.

`NumberFormatException` is in the `EnergyFragment.java`. It is Used when parsing the numeric values from strings. If the string format is invalid, this catch block prevents the app from crashing and likely defaults to a safe value or logs the issue.

27. Work on the feedback provided in the previous deliverables.
28. Complete Scrum dashboard, with all stories and tasks. Take screenshots of the last sprint only. Link for Monday: <https://legendsagencyadss-team.monday.com/boards/18390208856>

✓ Epic 1: Payment, Subscription, and UI/UX Refactor

Task		Owner	Status ⓘ	Due date ⓘ	+	
☐	✓ Story 1: Subscripti... 5	MA	Done	✓ Nov-30		
Subitem		Owner	Status	Date	Priority	+
☐	Task 1: Implement a Paym...	MA	Done	Nov 27	Medium	
☐	Task 2: Design and imple...	MA	Done	Nov 27	Medium	
☐	Task 3: Implement logic t...	MA	Done	Nov 28	Medium	
☐	Task 4: Develop a clear an...	MA	Done	Nov 30	Medium	
☐	Task 5: Implement Plan Fe...	MA	Done	Nov 30	Easy	
☐	+ Add subitem					

✓ Epic 2: Quality Assurance, Testing, and Architectural Refinement

Task		Owner	Status ⓘ	Due date ⓘ	+	
☐	✓ Story 1: Comprehe... 5	MA	Done	✓ Nov-30		
Subitem		Owner	Status	Date	Priority	+
☐	Task 1: Develop a strategy...	MA	Done	Nov 24	Medium	
☐	Task 2: Create an Espress...	MA	Done	Nov 24	Medium	
☐	Task 3: Develop three End...	MA	Done	Nov 25	Medium	
☐	Task 4: Create a Robolect...	MA	Done	Nov 29	Medium	
☐	Task 5: Implement three a...	MA	Done	Nov 30	Medium	
☐	+ Add subitem					

...	☐	▼ Story 2: Architectu...	5	✦ ⬆		Done	✓	Nov-30	
☐	Subitem				Owner	Status		Date	Priority
☐	☐	Task 1: Introduce the Pres...	⬆		J0	Done		Nov 24	Medium
☐	☐	Task 2: Refactor the Feed...	⬆		J0	Done		Nov 26	Medium
☐	☐	Task 3: Implement Air Qu...	⬆		J0	Done		Nov 27	Easy
☐	☐	Task 4: Implement a Brigh...	⬆			Done		Nov 29	Easy
☐	☐	Task 5: Update the Energ...	⬆			Done		Nov 30	Medium
☐	+ Add subitem								

...	▼ Epic 3: Intelligent Automation & Alert System	5 Tasks / 21 Subitems							
☐	Task				Owner	Status ⓘ		Due date ⓘ	+
☐	▼ Story 1: Smart Ligh...	4	✦ ⬆		J0	Done	✓	Nov-30	
☐	Subitem				Owner	Status		Date	Priority
☐	☐	Task 1. Implement Presen...	⬆		J0	Done		Nov 26	Medium
☐	☐	Task 2: Update setAutoLi...	⬆		J0	Done		Nov 28	Medium
☐	☐	Task 3: mplement cancel...	⬆		J0	Done		Nov 29	Medium
☐	☐	Task 4: Update LightRepo...	⬆		J0	Done		Nov 30	Medium
☐	+ Add subitem								

...	☐	▼ Story 2: After-Hou...	5	✦ ⬆	J0	Done	✓	Nov-30	
☐	Subitem				Owner	Status		Date	Priority
☐	☐	Task 1: Create Notificatio...	⬆		J0	Done		Nov 27	Medium
☐	☐	Task 2: Update Presence...	⬆		J0	Done		Nov 27	Medium
☐	☐	Task 3: Implement time-w...	⬆		J0	Done		Nov 28	Easy
☐	☐	Task 4: : Update Presence...	⬆		J0	Done		Nov 29	Easy
☐	☐	Task 5: Wire autoOffListe...	⬆		J0	Done		Nov 30	Medium

☐	▼ Story 3: Air Quality... 4	✦	+	JO	Done	✓	Nov-30
☐	Subitem			Owner	Status	Date	Priority
☐	Task 1: Update AirQuality...	+			Done	Nov 26	Medium
☐	Task 2: Implement a "Cool...	+			Done	Nov 28	Easy
☐	Task 3: Connect AirQualit...	+		JO	Done	Nov 29	Medium
☐	Task 4: Ensure the "Alert ...	+		JO	Done	Nov 30	Medium
☐	+ Add subitem						

...

▼

Story 4: In-App Ev...

5

JO

MA

Done

Nov-30

Subitem

Owner

Status

Date

Priority

Task 1: Design Notificatio...

MA

Done

Nov 25

Medium

Task 2: Create Notificatio...

JO

Done

Nov 26

Medium

Task 3: Update Notificati...

MA

JO

Done

Nov 27

Medium

Task 4: Refactor Notificat...

MA

Done

Nov 29

Medium

Task 5: Update Notificati...

JO

Done

Nov 30

Medium

+ Add subitem

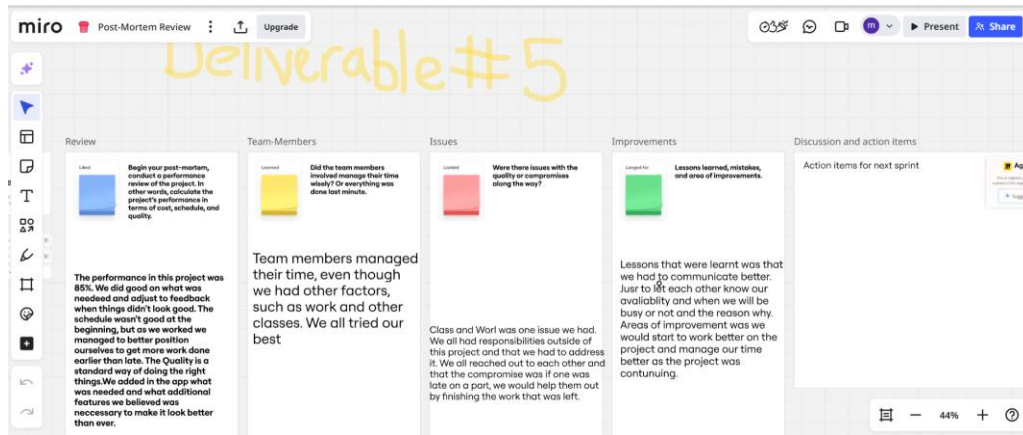
☐	▼ Story 5: Global "Ma... 3	✦	+	JO	Done	✓	Nov-30
☐	Subitem			Owner	Status	Date	Priority
☐	Task 1: Update PresenceV...	+		JO	Done	Nov 28	Medium
☐	Task 2: Ensure disabling t...	+		JO	Done	Nov 29	Medium
☐	Task 3: Verify that no bac...	+		JO	Done	Nov 30	Medium
☐	+ Add subitem						

29. Post-Mortem, Project Review Meeting, document the below:
- Begin your post-mortem, conduct a performance review of the project. In other words, calculate the project's performance in terms of cost, schedule, and quality.
 - Did the team members involved manage their time wisely? Or everything was done last minute.
 - Were there issues with the quality or compromises along the way?
 - Lessons learned, mistakes, and area of improvements.
 - Who attended and who missed the meeting.

30. Use a tool to record your Project Review Meeting (i.e.

<https://lucidspark.com/landing/create/online-sticky-notes>

<https://miro.com/app/board/uXjVJilEt6w=/>



Link for Recording for microsoft teams:

https://humberital.sharepoint.com/sites/LIFE9/_layouts/15/stream.aspx?id=%2Fsites%2FLIFE9%2FShared%20Documents%2FGeneral%2FRecordings%2FMeeting%20in%20General%2D20251130%5F215331%2DMeeting%20Recording%2Emp4&referrer=StreamWebApp%2EWeb&referrerScenario=AddressBarCopied%2Eview%2Ea94f9f89%2D37f7%2D48ca%2D8933%2Dde2c7854584d

31. How did you address technical debt in your project.: **We address technical debt by visiting the feedback from the previous deliverables and making sure that the deliverable feedback can be applied to this current deliverable. For example, in the previous deliverable, the feedback says to remove the about us from the navigation drawer to the overflow. We will do just that. We also are addressing technical debt by making the firebase contain our sensor readings from our hardware**
32. Document two areas of refactoring and why you did it!. Real examples from your code and not just statements. Copy the code from your project and comment on it. **The first area of refactoring is in the MainActivity.Java. The Refactoring happens when instead of repeating beginTransaction(), replace(), and commit() calls throughout the activity, the logic will extract it into a single helper method called setCurrentFragment. This was done because it reduces duplication. In the Navigation logic, it appears in multiple places (Drawer navigation, Bottom navigation, onCreate). Another reason is consistency. It ensures every fragment transaction handles the back stack consistently via the addToBackStack boolean. Lastly, Maintainability. If the container ID**

(R.id.main) or transition animation needs to change, you only update it in one place.

Code showing it

```
// Refactored method in MainActivity.java
private void setCurrentFragment(Fragment fragment, boolean
addToBackStack) {
    FragmentTransaction transaction =
fragmentManager.beginTransaction();
    transaction.replace(R.id.main, fragment);
    if (addToBackStack) {
        transaction.addToBackStack(null);
    }
    transaction.commit();
}

// Usage example (Clean and readable)
if (itemId == R.id.nav_settings) {
    setCurrentFragment(new SettingsFragment(), true);
} else if (itemId == R.id.dashboard) {
    setCurrentFragment(new DashBoardFragment(), false);
}
```

The second area of refactoring is in the DashBoardFragment.java. The code was refactored to fetch user data via an interface (UserDataProvider) rather than calling FirebaseDatabase directly inside the UI logic. The implementation FirebaseUserDataProvider is passed into the method. First reason why it was done was Separation of Concerns. The Fragment focuses on displaying the data while the Provider handles fetching it. Another reason is Testability. We will do unit tests to pass the class to verify the greeting logic without relying on a real Firebase instance. Lastly, flexibility. If the backend changes, you only need to create a new provider implementation without rewriting the Fragment.

Code showing it

```
// DashBoardFragment.java
protected void loadGreeting(UserDataProvider dataProvider) {
    dataProvider.fetchUserData(new UserDataProvider.UserDataCallback() {
        @Override
        public void onDataReceived(String userName) {
            // Update UI logic...
            setGreeting(firstName);
        }
        // ...
    });
}
```

}

// Usage in onCreateViewCreated
loadGreeting(new FirebaseUserDataProvider());

33. Describe how DevOps would have helped your project.: **DevOps would help in this project by having improved collaboration on the work. We break down responsibilities on each member and we do the required responsibility. Another is better quality and reliability. We work on a task, and if we find any errors or bugs we immediately fix it so that when doing code review, we have no problems in our product. This is also associating in making sure we have faster delivery on the app. DevOps allow quick feedback on the project and this can help us respond where we went right and where we went wrong.**
34. What coding standards did you use, and how did you use them.: **1. Naming Conventions You generally follow standard Java naming conventions, but there are notable exceptions. These include Classes, Methods & Variables, Constants, Layouts & Resources. 2. Architecture & Design Patterns: in our project we use a mix of MVC (Model-View-Controller) and Component-Based architecture tailored for Android. •Fragments as Controllers: Logic is heavily centered in Fragments which handle both UI updates and business logic Separation of Concerns. 3. Libraries & Dependencies: in the build.gradle and imports we indicate specific standard library choices. These include Material Design, Modern Android Jetpack libraries. 4. Testing Standards •Frameworks: You use JUnit 4 for unit testing and Espresso for UI testing. •Naming: Test methods follow the testFeatureName convention (e.g., testConstructorInitializesFields in HelperClassTest). •Assertions: You use standard JUnit assertions (assertEquals, assertNotNull). 5. Formatting & Style •Annotations: You consistently use @NonNull and @Nullable to prevent NullPointerExceptions (e.g., in onCreateView and onDataChange). •Indentation: Standard 4-space indentation is used across Java files. •Logging: Usage of Toast for user feedback and Log (implied or sparse) for debugging.**
35. What AI tool used and how it helped in your work. **The AI tool we used is Gemini. Gemini in this project has help us in making code. If i go to gemini on Android Studio, i can ask to make unit tests. And when i run the unit test, if it doesn't work to my expectations i tell it to do it again. Gemini also allowed us to debug our work. If we keep getting errors we can paste the code in a snippet and we can ask it to debug. Troubleshooting is another way gemini has help us out. We can paste in the class or the structure of the project and it can go through it. Lastly, it can do tasks (e.g 10 tasks) and gemini will lay down the outline and the structure for the 10 tasks and we will see the design on how to do the tasks and the code showing the UI.**

36. List at least one security vulnerability in your code, copy the code and comment on it. How to address it in the future.: **The helper class is used as a Data Transfer Object to save user information to the Firebase Realtime Database. The vulnerability is the data model is directly including the fields for the user's text password. Also, storing or transmitting the plain text passwords, even momentarily in a data class, greatly increases the risk of a security breach through accidental logging, memory dumps, or incorrect database write operations.**

What we can do in the future is to Remove Password Fields from the Data Model. The fields password and confirmPassword should be removed from HelperClass.java. The class should only store public/non-sensitive profile information. We can also ensure we have secure authentication logic in our app. This helps by having the application rely only on the Firebase Authentication service for password handling. Firebase secures hashes and stores passwords, preventing the developers or applications from accessing the passwords. The HelperClass should only be used to create the user's profile entry in the database *after* a successful authentication call.

Code to show the Vulnerability in the helperClass:

```
package ca.light.indoorair.freshness.energy.it.life.environmental.solution;  
  
public class HelperClass {  
  
    // VULNERABILITY: 'password' and 'confirmPassword' should not be  
part of the  
    // user profile model stored in the database.  
    String name, email, phoneNumber, password, confirmPassword;  
  
    public String getName() {  
        return name;  
    }  
  
    // ... (Getters and Setters) ...  
}
```

Code to fix in the future

```
public class HelperClass {  
    // Only store non-sensitive profile data  
    String name, email, phoneNumber;  
  
    public HelperClass(String name, String email, String phoneNumber) {  
        this.name = name;  
    }
```

```

        this.email = email;
        this.phoneNumber = phoneNumber;
    }
    // ...
}

```

Create a table showing some analytics about using your app: by all users (Not a single user)

	Last 24 hrs	Last 7 days
Home Screen	25 times	75 times
Presence Screen	8 times	15 times
Energy Screen	20 times	40 times
Light Screen	30 times	50 times
Purchases screen	40 times	55 times
Feedback screen	5 times	15 times
About Us screen	20 times	35 times
Settings Screen	20 times	50 times

37. Describe your solution to pull the statistic above:

We achieve these analytics by integrating Google Analytics for Firebase into our application. We instrumented a manual `logScreenView` event within the `onResume()` lifecycle method of each Fragment (Home, Presence, Energy, etc.).

When a user navigates to a screen, the app triggers a `SCREEN_VIEW` event with the specific screen name parameter. These events are batched and uploaded to the Firebase servers.

38. Provide Screenshots showing the solution you used.

```
build.gradle (LIFE_EnvironmentalSolution)  build.gradle (:lifeenvironmentalsolution)  MainActivity.java
You can use the Project Structure dialog to view and edit your project configuration  Open (Ctrl+Alt+Shift+S)  Hide notification

40  dependencies {
41      implementation libs.preference
42      testImplementation libs.junit
43      testImplementation 'org.robolectric:robolectric:4.11.1'
44      testImplementation libs.core
45      androidTestImplementation libs.ext.junit
46      androidTestImplementation libs.espresso.core
47
48      // Import the BoM for the Firebase platform
49      implementation(platform("com.google.firebase:firebase-bom:34.3.0"))
50
51
52      implementation("com.google.firebase:firebase-auth")
53
54          implementation("androidx.credentials:credentials:1.3.0")
55      implementation("androidx.credentials:credentials-play-services-auth:1.3.0")
56      implementation("com.google.android.libraries.identity.googleid:googleid:1.1.1")
57
58      //Lottie
59      implementation("com.airbnb.android:lottie:6.4.1")
60
61
62      implementation platform('com.google.firebase:firebase-bom:32.7.0')
63
64      implementation 'com.google.firebase:firebase-analytics'
65  }
```

```

61 public class MainActivity extends AppCompatActivity implements FragmentManager.OnBack
    7 usages
66 private DrawerLayout drawerLayout;
    4 usages
67 private NavigationView navigationView;
    5 usages
68 private ActionBarDrawerToggle toggle;
    17 usages
69 private BottomNavigationView bottomNavigationView;
70
    2 usages
71 private static final String PREFS_NAME = "MyPrefsFile";
    2 usages
72 private static final String THEME_KEY = "ThemeKey";
    1 usage
73 private FirebaseAnalytics mFirebaseAnalytics;
74
75 @SuppressWarnings("ClickableViewAccessibility")
76 @Override
77 protected void onCreate(Bundle savedInstanceState) {
78     super.onCreate(savedInstanceState);
79     setContentView(R.layout.activity_main);
80
81     mFirebaseAnalytics = FirebaseAnalytics.getInstance(context, this);
82     setupWindowInsets();
83     setupToolbar();
84
85     auth = FirebaseAuth.getInstance();
86     fragmentManager = getSupportFragmentManager();
87
88     fragmentManager.addOnBackStackChangeListener(this);
89

```

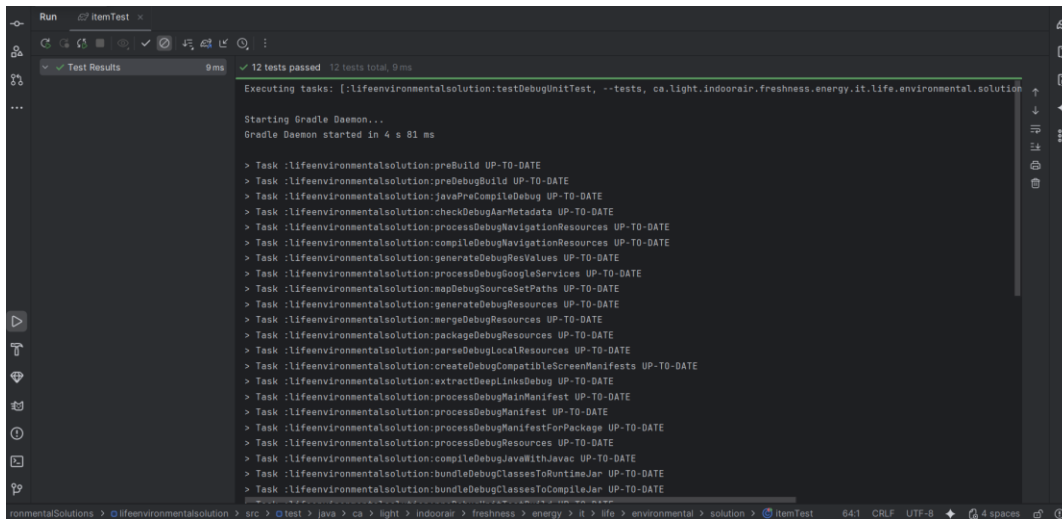
```

onmentalsolution)  DashboardFragment.java  PresenceFragment.java  AirQualityFragment.java x
29  public class AirQualityFragment extends Fragment {
60      public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
82
83          setupControllListeners();
84          observeViewModel();
85          loadSettings();
86      }
87
88
89      no usages
90      private String getSelectedRoomFromArguments() {
91          Bundle args = getArguments();
92          if (args != null && args.containsKey("SELECTED_ROOM_KEY")) {
93              return args.getString(key: "SELECTED_ROOM_KEY", defaultValue: "Main Office");
94          }
95          return "Main Office";
96      }
97
98      @Override
99      public void onResume() {
100          super.onResume();
101          // Log that the user is viewing the Presence Screen
102          FirebaseAnalytics mFirebaseAnalytics = FirebaseAnalytics.getInstance(requireContext());
103          Bundle bundle = new Bundle();
104          bundle.putString(FirebaseAnalytics.Param.SCREEN_NAME, "Presence Screen");
105          bundle.putString(FirebaseAnalytics.Param.SCREEN_CLASS, "PresenceFragment");
106          mFirebaseAnalytics.logEvent(FirebaseAnalytics.Event.SCREEN_VIEW, bundle);
107
108      }
109
110      1 usage
111      private void initializeViews(View view) {
112          airQualityProgress = view.findViewById(R.id.air_quality_progress);

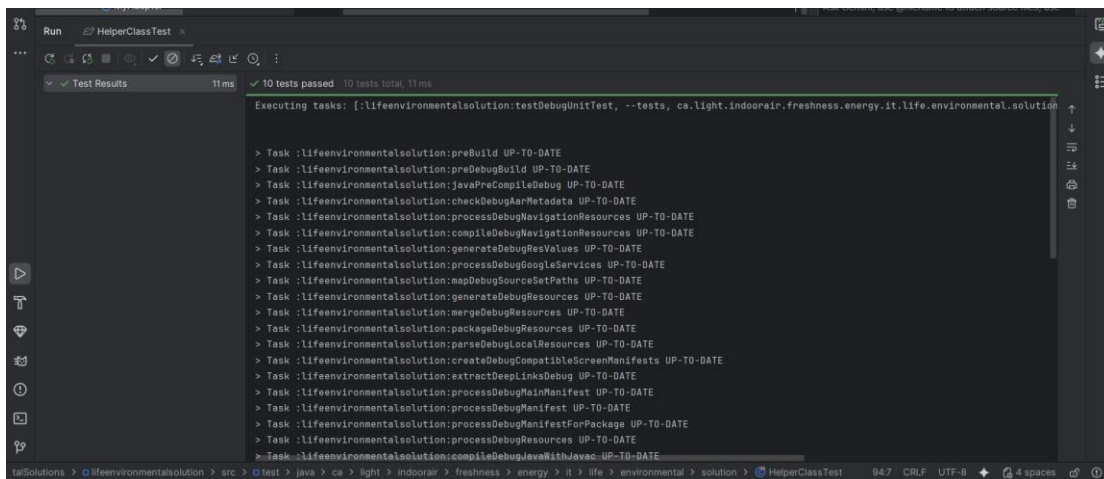
```

39. Describe your strategy for writing the test cases.: **our strategy for the test cases was first Local Unit Tests (JVM Tests).** These tests run on the local machine's Java Virtual Machine (JVM) and are fast. They are used to verify business logic and data models without needing an Android emulator or device. The next strategy is **Instrumented Tests (UI Tests).** These tests run on a hardware device or emulator. They verify that the application behaves correctly when a user interacts with it. •Location: src/androidTest/java/.Framework: Espresso •Strategy: •User Flows: You test complete user journeys, such as the Sign-Up process. •Example: SignupActivityEspressoTest likely fills out the sign-up form, clicks the button, and verifies the result. •Context Verification: Simple tests like ExampleInstrumentedTest ensure the app Context is initialized correctly.

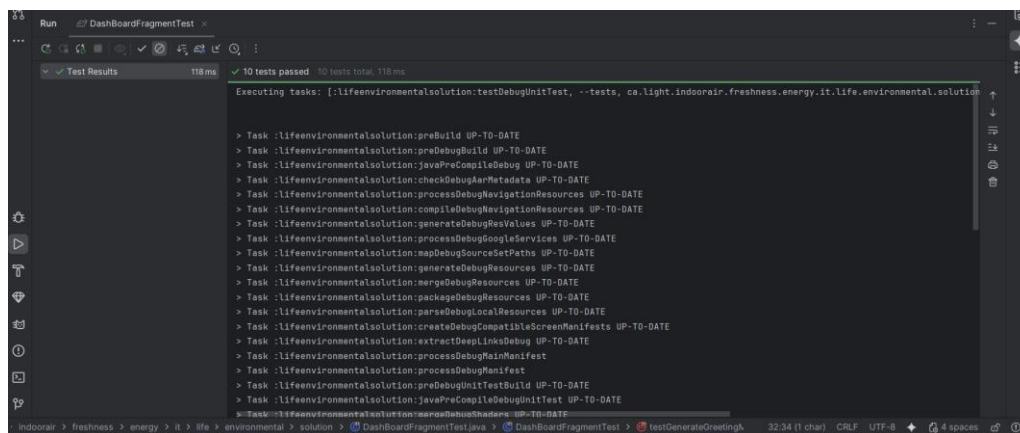
40. Screenshots showing 3 classes junit test cases are passing.
Screenshot for itemTest.Java



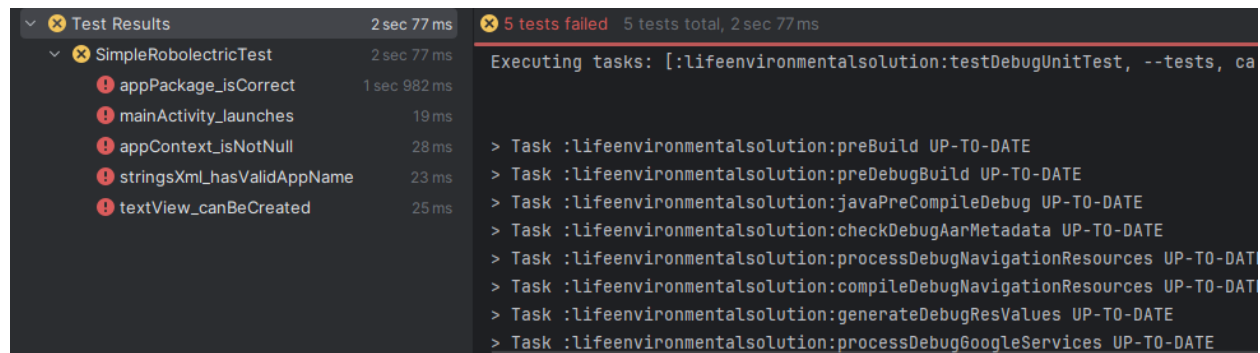
ScreenShot for HelperClassTest.Java



Screenshot showing DashBoardFragment.Java

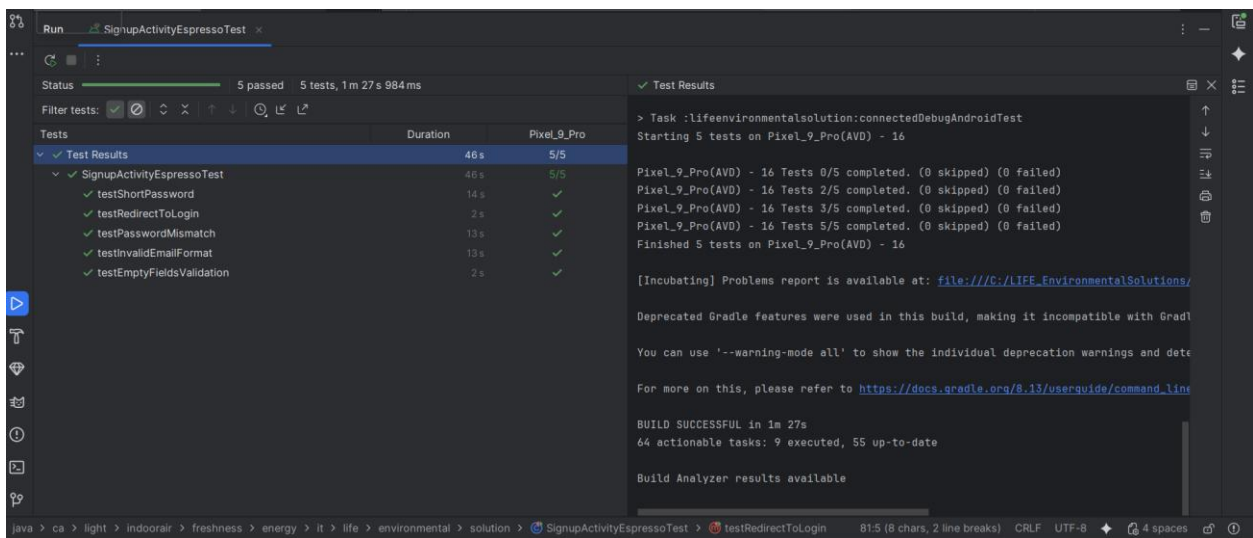


41. Screenshot showing Robleteric test cases are passing



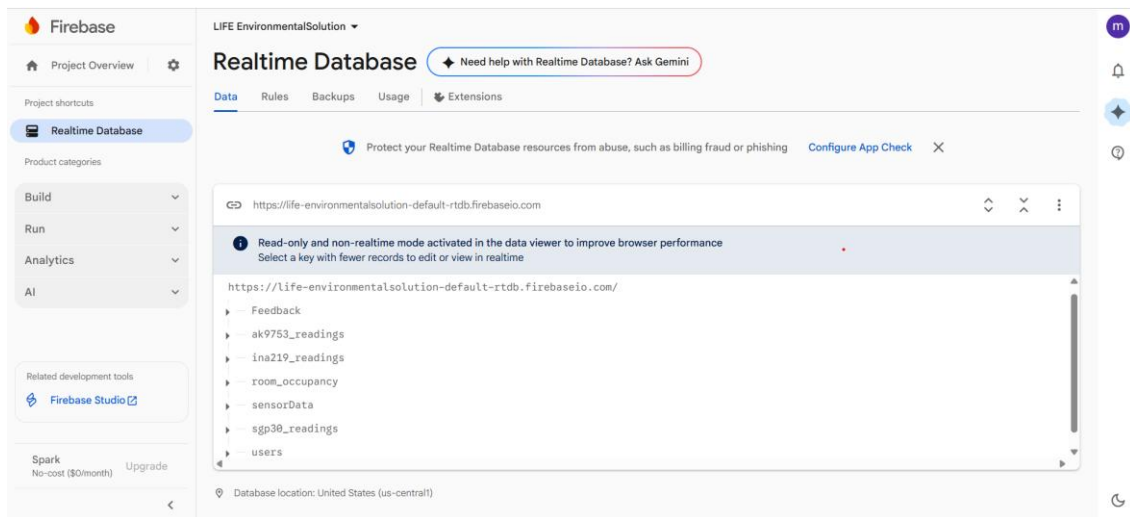
Are not passing because project runs on sdk 36 and roboelectric only

42. Screenshot showing Espresso test cases are passing.
Screenshot for SignUpActivityEspresso.Java

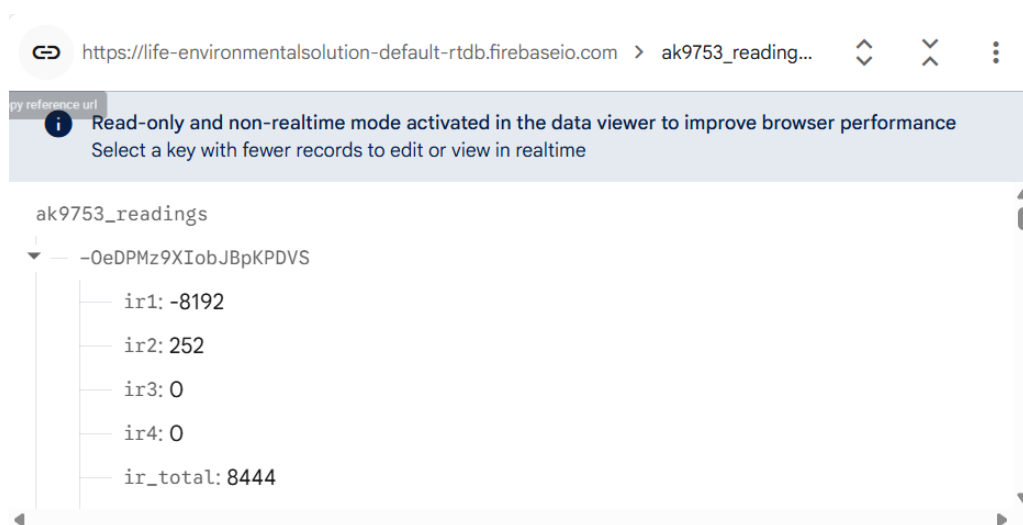


44. Screenshots showing the data in your DB, sensor data (at least 3 different sensors), and login data (including Google login).

Screenshot showing data in DB



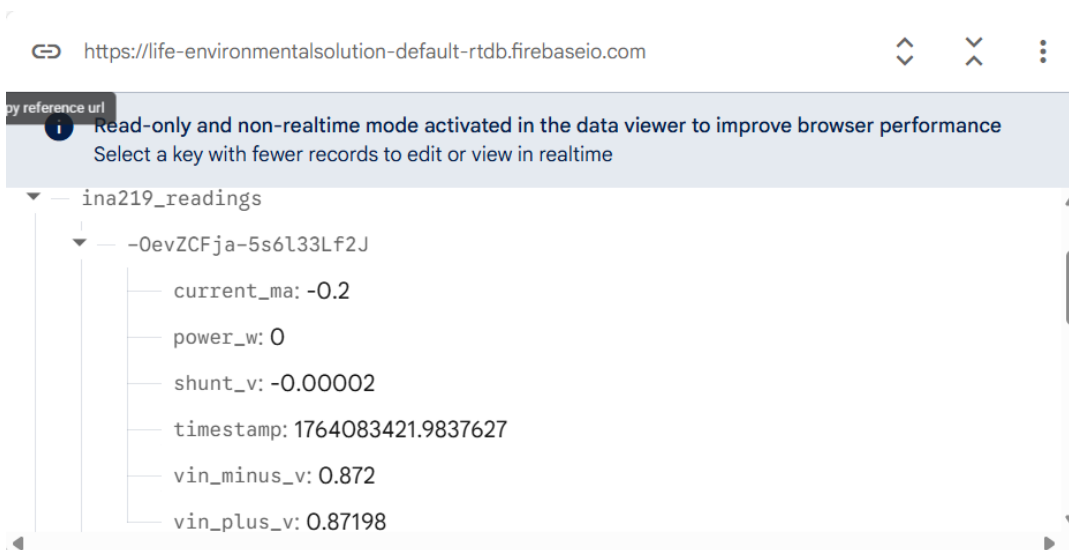
Screenshot showing the ak9753_readings



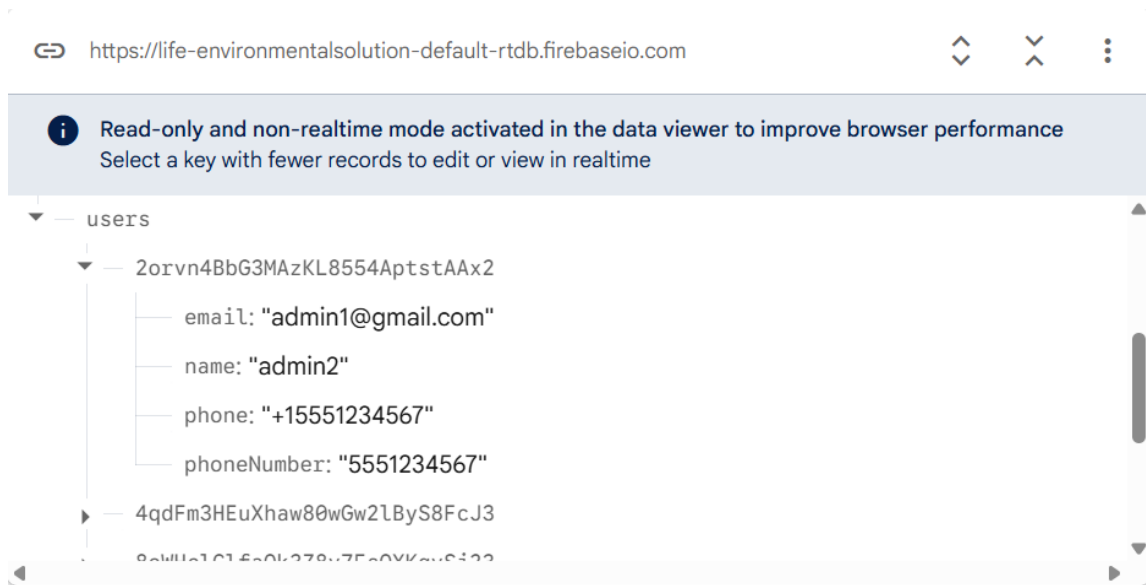
Screenshot showing the sgp30_readings



Screenshot showing ina219_readings



Screenshot showing the Login Data



45. Add into pdf file screenshot showing the data stored into the DB from Feedback. Must have at least 3 different entries.

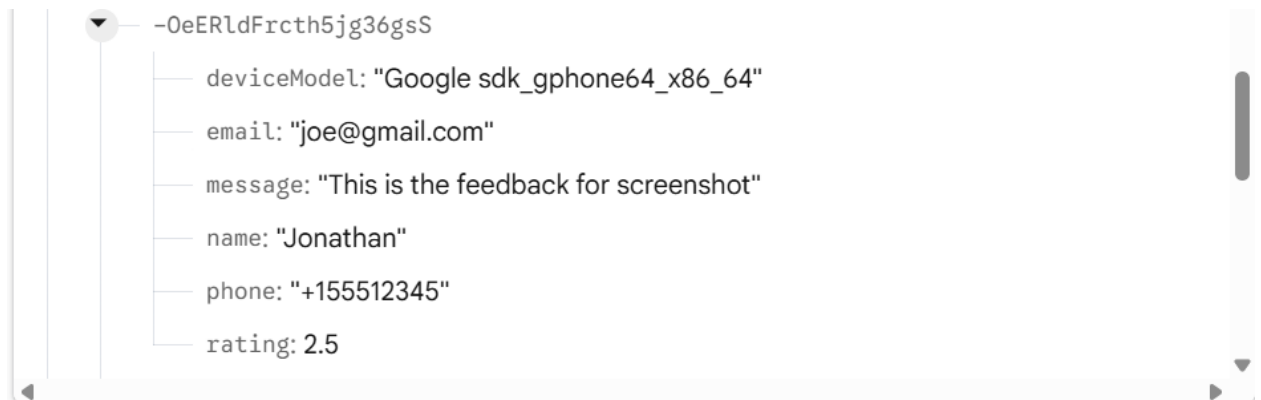
Screenshot For feedback Data #1



Screenshot for feedback Data #2



Screenshot for feedback Data #3



46. Suggestions to the instructor for future projects, things you liked, and things you suggest to be done differently and how.: **Future projects can have different kinds of ideas. We don't need to think about ideas that have to be smart technology, rather things we can do or other ventures that can be done on app. For example, showing a company that deals with cars, or clothing or something that people can enjoy and that are essential to daily living. We can also do creative learning, like an app that can demonstrate tools that can better students in education. For example, a tutorial for math or different kinds of experiments to try at home like a volcano using baking soda and vinegar.**

47. Create a subfolder called deliverable 5 under docs in the repo and add the pdf file.
 48. Commit and push the ppt into deliverable 5 folder, this is the only file that can be committed and pushed pass the due and before your presentation.
 49. All members must present. Zero for members who miss the presentation.
 50. Practice on presenting, and be ready, each team member will have ¹ - ² minutes to present.
 51. If document not in pdf, or not in the proper name (marks will be deducted!)
Please save the PDF document, i.e.
TeamName_ProjectName_Deliverable³.pdf
 52. One member to submit the pdf file.
-

Presentation Material

- 6- Demo of the app and flows, and the data on the different screens, focus on the functionality of your app, the **area which is different than other apps**.
 - 7- Show backend DB, how the data stored and retrieved.
 - 8- Demo a change on the DB, reflected on the app.
 - 9- Demo a change in the app, reflected on the DB.
 - 10- Offline mode.
 - 11- Analytics incorporated in the app.
 - 12- Explain test cases code, focusing on Junit, and high level for Robletric and Espresso
 - 13- Sprint dashboard on the website.
 - 14- All members must present. Zero for members who miss the presentation.
 - 15- Practice on presenting, and be ready, each team member will have 3 - 4 minutes to present.
 - 16- For things to run faster, have a tab opened for the DB and the scrum dashboard website. Have your Android studio running and the app pushed into the simulator.
-

¹ - Power Point highlights the main components, focus on what **distinguish** your apps compared to other apps.

² - Why is your project unique compared to other projects in this course.

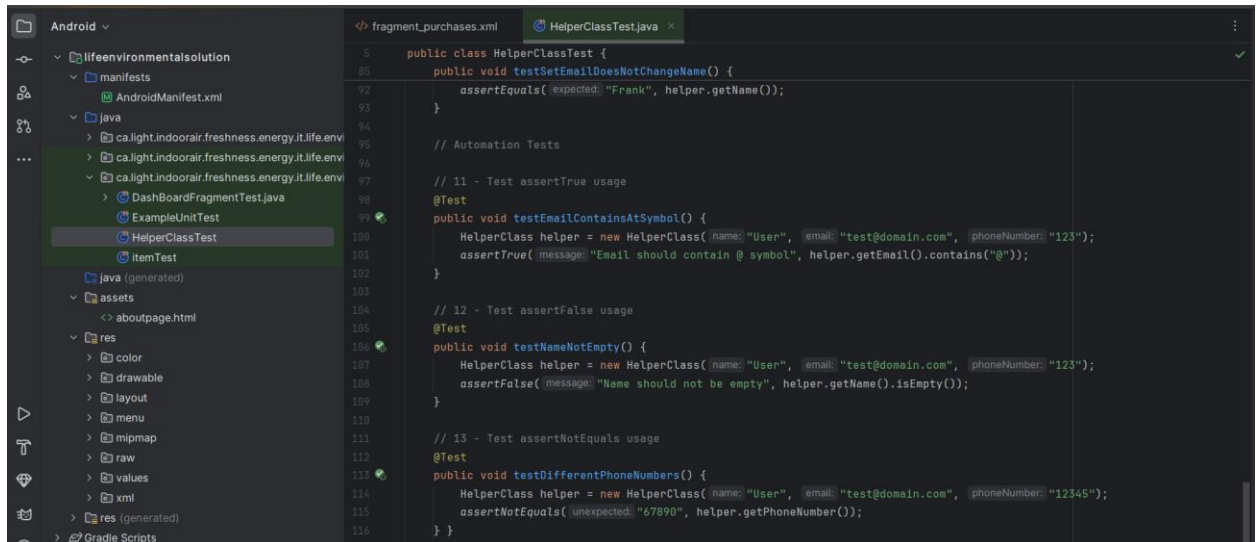
³ - Why is your app unique compared to other apps in the market, which have similar functionality.

Deliverable 5:

Android Studio Project pushed into in github:

- 1- Use Navigation Drawer with the combination with another layout (Bottom Navigation and TabsLayout).
- 2- Complete the UI design. All the UX must be completed.
- 3- No hardcoding of data. All data is read from and written to DB on the cloud.
- 4- All functionality is complete.
- 5- Should implement a functionality for off-line mode.
- 6- Should have a minimum of 10 java classes.
- 7- Each member must have a minimum of 20 commits. Counted starting Nov. 17.
- 8- **Expect commits for at least 8 days (Nov. 17 - 30) for all members.** The work should be done over the sprint, and not in the last 48 hours prior to the submission.
- 9- Must have meaningful commits.
- 10- Implement animation for the Splash screen, and two different splash screens (Dark and Light). Take screenshots showing the two different splash screens.
- 11- All functionality on all screens have been completed.
- 12- No Work in progress.
- 13- All data retrieved and pushed into the DB on the cloud.
- 14- All requirements requested in previous deliverables have been completed.
- 15- Complete all the requirements, the application must be ready to be handled to the customer.
- 16- Application must be fully functional, and no remaining ToDo feature.
- 17- Demonstrate the use of Fab button on one of the screens, with some proper functionality:
<https://material.io/components/buttons-floating-action-button>
- 18- Write 3 types of Automation test cases: Refer to Week 9 on how to write test cases.

Screenshot showing the Automation tests insideHelperClassTest.java



- a. At least two different test classes using Junit (unit tests) and minimum of 10 test cases per class. Must use assertTrue, assertFalse, assertEquals, assertNotEquals and assertNotNull. Refer to Calculator and InputValidation examples.
 - b. These two classes to be tested, must be different than one you did in deliverable 4 (in total there must be 3 different test junit classes, to test 3 different classes in your app).
 - c. At least one test class using Robolectric and minimum of 5 test cases. Test a class different than the ones in b.
 - d. At least one test class using Espresso and minimum of 5 test cases. Test a class different than the ones in b & c.
 - e. Test valid and invalid inputs, i.e. valid email aaa@bbb.com, and invalid email aaa@
 - f. All test cases must pass.
 - g. All test cases must be meaningful and not just a repeat.
- 19- All functionality and connectivity to DB is complete.
 - 20- Plus deliverable 1 + 2 + 3 + 4.
 - 21- App must not have compilation errors.
 - 22- App must not crash.
 - 23- App will be tested on different simulators and devices.
 - 24- Must not have an option on the Settings screen to switch Language. Language is set at the device level and not at the app level.
 - 25- Remove all commented out code.
 - 26- Clean up the code. Have proper format and naming convention.

----- **Deliverable 4:**

Refer to deliverable 4.